

SAKYTI Multilingual NLP Architecture

End-to-End Multilingual Healthcare NLP, Ayurvedic Terminology Shielding & IndicTrans2 Neural Pipeline

1. Executive Summary & Architectural Overview

The **SAKYTI Multilingual NLP Service** is a production-grade asynchronous microservice developed using **FastAPI**. It provides high-precision natural language understanding across **22 official Indic languages**, detects and transliterates mixed-script **Romanized (Hinglish)** input, protects classical **Ayurvedic clinical terminology** (e.g., *Vata, Pitta, Kapha, Agni, Ama*) from literal machine translation corruption, and provides bidirectional neural translation with **AI4Bharat IndicTrans2** models.

CLIENT LAYER	Web UI, Mobile Apps, Clinical Chatbots, Diagnostic Triage Tools sending REST payloads with optional X-Request-ID.
GATEWAY & MIDDLEWARE	FastAPI ASGI application with RequestIDMiddleware (contextvars correlation), CORSMiddleware, and strict Pydantic v2 schema validators.
DOMAIN SERVICES LAYER	• Normalizer: Unicode NFC, whitespace, punctuation & danda (■) • IndicLID: FastText dual-model language & script identification • Romanized Engine: Hinglish lexical detection & term-shielded transliteration • Terminology Service: 14 core Ayurvedic concepts protection • Translator: Pure Python IndicProcessor + IndicTrans2 Transformer
MODELS & KNOWLEDGE BASES	• IndicLID-FTN / IndicLID-FTR (.bin FastText classifiers) • IndicTrans2-indic-en-dist-200M / en-indic-dist-200M (RoPE Seq2Seq) • data/ayurveda_terms.json (Classical taxonomy & provenance) • data/languages.json (Canonical language registry with Sanskrit)
DOWNSTREAM ECOSYSTEM	SAKYTI Diagnostic LLM, Clinical Knowledge Graph, and Ayurvedic Electronic Health Records (EHR).

HIGH-LEVEL DATA FLOW

User Query [e.g. '*mujhe pet me dard hai aur pitta vikriti lagti hai*']
▼
FastAPI Gateway [Request-ID Injection → Pydantic Validation → Router Dispatch]
▼
SAKYTI 7-Stage NLP Pipeline [Normalize → IndicLID → Romanized Handle → Term Extract → IndicTrans2 → IR]
▼
Standard Intermediate Representation [Language: Hindi | Script: Latin/Romanized | Terms: Pitta, Vikriti | English: '*I have abdominal pain and feel Pitta Vikriti*']

2. FastAPI Request Lifecycle & Internal Mechanics

This section details how an HTTP request enters the microservice, gets transformed through ASGI middlewares, undergoes Pydantic validation, executes through asynchronous lifecycles, and formats structured error responses.

Step / Component	Internal Mechanics & Behavior	Error Handling / Guardrails
1. ASGI Server (Uvicorn)	Receives raw TCP connection, parses HTTP/1.1 or WebSocket protocol, forwards scope, receive, send ASGI events.	Rejects malformed HTTP packets; manages async event loops.
2. RequestID Middleware	Inspects X-Request-ID header. If missing, generates a cryptographic UUIDv4. Binds it to Python contextvars and request.state.request_id.	Guarantees 100% request correlation across all log statements and response headers.
3. CORS Middleware	Evaluates request Origin against configured allowlist. Handles HTTP OPTIONS preflight requests immediately.	Blocks unauthorized cross-origin requests in production.
4. Route Matcher & Dependencies	Resolves path against APIRouter (e.g. POST /api/v1/nlp/analyze). Evaluates FastAPI dependencies (db sessions, settings).	Returns standard 404 Not Found or 405 Method Not Allowed with structured envelope.
5. Pydantic v2 Schema Validator	Deserializes JSON payload into typed model (e.g. NLPAnalyzeRequest). Enforces text length (≤5,000 chars), non-emptiness, thresholds.	HTTP 422 Unprocessable Entity: Caught by validation_exception_handler returning RFC-7807 ErrorResponse.
6. Lifespan Model Singletons	FastAPI @asynccontextmanager lifespan loads IndicLID FastText and IndicTrans2 PyTorch weights once at boot. Avoids per-request re-initialization.	Detects CUDA availability; falls back to CPU automatically. Models are pinned to memory.
7. Service Execution	Dispatches request to SAKYTIMultilingualPipeline. Employs async executors or non-blocking threads for model inference.	Any unhandled exception is caught by global exception handler, logging stacktrace and returning HTTP 500 .
8. Response Formatting & Headers	Serializes Pydantic response model to JSON. Middleware intercepts outgoing response, injecting X-Request-ID header.	Client receives deterministic JSON with latency telemetry (execution_time_ms).

FastAPI Unified Error Envelope Architecture

All HTTP exceptions (400, 404, 413, 422, 500, 503) are caught by custom exception handlers registered in app/main.py. They return a strictly structured JSON envelope matching the ErrorResponse Pydantic schema:

```
{ "error": { "code": "VALIDATION_ERROR", "message": "Request payload validation failed.", "details": [{"loc": ["body", "text"], "msg": "Input should be a valid string", "type": "string_type"}], "request_id": "c9b1f2a3-8d4e-4b6a-9f1c-7e5d2a8b3c1d", "timestamp": "2026-09-08T18:30:00Z" } }
```

3. SAKYTI 7-Stage Multilingual NLP Pipeline

The core linguistic intelligence engine executes across seven sequential, decoupled stages inside services/pipeline.py. Each stage transforms the input query while maintaining immutability and provenance tracking.

Stage	Subsystem & Method	Detailed Functionality & Transformation
Stage 1: Input Validation	Pipeline._validate_input (Task 1 & 8)	• Verifies non-empty string input and character length limits ($\leq 5,000$ characters). • Sanitizes control characters and null bytes. • Protects downstream PyTorch models from GPU memory exhaustion.
Stage 2: Multilingual Normalization	services/normalizer.py (Task 6)	• Applies Unicode NFC canonical decomposition/recomposition. • Decodes HTML entities and normalizes whitespace. • Collapses repeated punctuation (e.g. '!!!!' $\rightarrow$ '!'). • Crucial: Preserves sacred Indic punctuation including Devanagari Danda (■), Double Danda (■■), and medical abbreviations.
Stage 3: Language & Script ID	services/language_detector.py (Task 2 & 3)	• Inspects Unicode script blocks. • Routes native Indic text to IndicLID-FTN (22 languages). • Routes Latin/Romanized text to IndicLID-FTR to detect Romanized Indic vs English. • Returns canonical language code, script, and confidence.
Stage 4: Romanized Input Handling	services/romanized_processor.py (Task 7)	• Evaluates Romanized Indic confidence using lexical markers ('dard', 'pet', 'hai'). • Transliterate Romanized text to native Devanagari. • Shields known Ayurvedic terms so phonetic transliteration does not alter medical spellings. • Produces converted text for subsequent pipeline stages.
Stage 5: Ayurvedic Term Extraction	services/terminology_service.py (Task 5)	• Scans text against data/ayurveda_terms.json using boundary-aware regex. • Extracts character spans, Sanskrit forms, categories (Dosha, Dhātu, Agni, Herbology). • Tags concepts with do_not_translate: true for translation protection.
Stage 6: IndicTrans2 Translation	services/translator.py (Task 4)	• If target language differs from detected language (typically Indic $\rightarrow$ English): 1. Masks Ayurvedic terms with unique placeholders (__AYUR_TERM_0__). 2. Attaches IndicTrans2 language prefix tags (e.g. __hin_Deva__). 3. Runs AI4Bharat RoPE Transformer Seq2Seq inference. 4. Detokenizes and restores canonical clinical terms into target syntax.
Stage 7: IR Assembly & Output	Pipeline.process_query (Task 8 & 9)	• Synthesizes standardized Intermediate Representation (IR). • Bundles original text, normalized text, detected language, confidence, extracted terms with offsets, final English text, and execution latency.

4. Machine Learning Models & Ayurvedic Shielding

A rigorous, technical breakdown of the machine learning architectures, tokenization mechanics, and Ayurvedic clinical shielding.

4.1 IndicLID Language & Script Identification (AI4Bharat)

IndicLID utilizes subword n-gram fastText architectures trained across 22 official Indian languages and English. To avoid cross-script confusion between native scripts and Romanized text, our service employs a script-gated dual-model router: Unicode code-point analysis directs native Indic characters (U+0900 to U+0DFF) to IndicLID-FTN and Latin characters (U+0041 to U+007A) to IndicLID-FTR. This prevents English from being misclassified as an Indic language and vice-versa.

4.2 IndicTrans2 Transformer Translation Engine

IndicTrans2 is a 200M/1B parameter sequence-to-sequence Transformer architecture incorporating Rotary Position Embeddings (RoPE). It supports high-fidelity translation across all 22 scheduled Indian languages plus English and Sanskrit.

Feature / Component	Implementation Details in SAKYTI Service
Pure Python IndicProcessor	Original AI4Bharat tooling depended on compiled C++ Cython extensions. We authored a 100% pure Python implementation (app/utils/indic_processor.py) providing exact tokenization and script tagging parity with zero native compiler dependencies.
Language Tag Prefixing	Prepends target and source language tags (e.g. __hin_Deva__, __tam_Taml__, __eng_Latn__) at the token boundary, conditioning the decoder to generate grammatically correct target text.
Device Orchestration	Pytorch models are loaded onto CUDA GPUs if available; otherwise dynamically pin to CPU. Memory is allocated once at startup via the FastAPI Lifespan context manager.

4.3 Ayurvedic Terminology Shielding Mechanism

Generic machine translation models corrupt ancient Ayurvedic concepts by translating them literally into secular English terms. The SAKYTI Terminology Layer eliminates this risk via a 4-step preservation protocol:

Step 1: Term Matching	Aho-Corasick & boundary-aware regex matches canonical terms, Devanagari, and Romanized spellings from data/ayurveda_terms.json.
Step 2: Placeholder Masking	Matches are substituted with alphanumeric collision-proof boundary tokens: '■■■■ ■■■■' → __AYUR_TERM_0__ dosha.
Step 3: Neural Translation	IndicTrans2 translates the masked sentence. The transformer treats the placeholder as an opaque proper noun.
Step 4: Canonical Restoration	Placeholders are mapped back to their standardized Sanskrit / English canonical representations: __AYUR_TERM_0__ → Vata.

Concept	Sanskrit / Hindi	Category	Clinical Definition & Erroneous Literal Translation
Vata	■■■■ (v■ta)	Dosha	Biological kinetic principle. <i>Never translate as 'Air' or 'Wind'.</i>
Pitta	■■■■■■ (pitta)	Dosha	Biological metabolic & thermal principle. <i>Never translate as 'Bile'.</i>
Kapha	■■■ (kapha)	Dosha	Biological structural & cohesive principle. <i>Never translate as 'Phlegm'.</i>
Agni	■■■■■■ (agni)	Physiology	Digestive and metabolic fire. <i>Never translate as physical 'Fire'.</i>
Ama	■■■ (■ma)	Pathology	Endogenous metabolic toxins / undigested material.
Triphala	■■■■■■■■ (triphala■)	Herbology	Synergistic 3-myrobalan formulation. <i>Never translate as 'Three fruits'.</i>
Panchakarma	■■■■■■■■■■ (pañcakarma)	Therapy	Five classical bio-cleansing therapies. <i>Never translate as 'Five actions'.</i>

5. Production API Reference & Observability

The microservice exposes a standardized REST interface adhering to OpenAPI 3.1 standards. Every endpoint integrates automatic Pydantic request validation, response serialization, and distributed tracing headers.

Method & Route	Primary Purpose	Payload & Key Response Fields
POST /api/v1/nlp/analyze	Master 7-stage NLP Pipeline execution for full medical query analysis.	In: text, target_language, preserve_terms Out: Normalized text, detected language, confidence, script, Romanized status, extracted Ayurvedic concepts, translated English text, latency.
POST /api/v1/nlp/translate	Direct neural translation between Indic languages and English with terminology shielding.	In: text, source_language, target_language, preserve_ayurvedic_terms Out: translated_text, model version, device, execution time.
POST /api/v1/nlp/detect-language	Identifies language and script across 22 Indic orthographies & Romanized variants.	In: text, confidence_threshold Out: language_code, language_name, script, is_romanized, raw model label.
GET /health	Liveness, readiness, and model weight initialization status.	Out: Microservice version, environment, model status (IndicLID, IndicTrans2), device telemetry (CPU/CUDA).

5.1 Telemetry, Logging & Observability Architecture

- **Correlation Tracking (X-Request-ID):** Every inbound request receives or generates an X-Request-ID header. This ID is injected into Python's async contextvars, propagating through all logging records and API responses.
- **Structured JSON Logging:** Production logs are emitted as single-line JSON objects containing timestamp, level, request_id, message, and latency_ms, enabling seamless ingestion into Datadog, Grafana Loki, or Google Cloud Logging.
- **Test Coverage & Verification:** The service includes **175 automated unit and integration tests** validating Unicode normalization, regex edge cases, Romanized transliteration, IndicTrans2 translation accuracy, terminology preservation, and FastAPI HTTP status codes.

5.2 Complete Language Matrix (Initial Core Subset)

Language	Code	IndicTrans2 Tag	Native Script	Domain Role
Hindi	hi	hin_Deva	Devanagari	Primary User-Facing & Knowledge
Sanskrit	sa	san_Deva	Devanagari	Ayurvedic Source & Knowledge Language
Tamil	ta	tam_Taml	Tamil	User-Facing Language (Siddha/Ayurveda)
Telugu	te	tel_Telu	Telugu	User-Facing Language
Bengali	bn	ben_Beng	Bengali	User-Facing Language
Marathi	mr	mar_Deva	Devanagari	User-Facing Language
Gujarati	gu	guj_Gujr	Gujarati	User-Facing Language
Kannada	kn	kan_Knda	Kannada	User-Facing Language
Malayalam	ml	mal_Mlym	Malayalam	User-Facing Language (Kerala Ayurveda)
Punjabi	pa	pan_Guru	Gurmukhi	User-Facing Language
Odia	or	ory_Orya	Odia	User-Facing Language
English	en	eng_Latn	Latin	Pivot Translation & Clinical Model Target